Nearby Services WebGIS for Pune

Name: Aditya Singh
PRN: 25070243005
Course: M.Sc. DSSA 2025-27

1. Introduction

This project is a web-based GIS that helps a user find essential services near a chosen location in Pune, Maharashtra. The user picks a spot on the map, types coordinates, or uses browser geolocation. They then select service types such as hospitals or pharmacies and receive a list of nearby places sorted by distance. Every result is also drawn on an interactive map, with a popup and a details panel.

The goal was to build a complete spatial pipeline: a spatial database that answers the distance queries, a map server that publishes the data as standard OGC services, and a web frontend that makes the results easy to use.

2. System Architecture

The system has four layers. OpenStreetMap data is loaded into PostGIS. GeoServer publishes the data as WMS and WFS layers. A Leaflet web page sends requests to GeoServer and shows the answers.

Component	Technology	Role
Data source	OpenStreetMap through the Overpass API	Supplies service locations and attributes
Database	PostgreSQL with PostGIS	Stores points and runs the spatial queries
Map server	GeoServer	Publishes layers through WMS and WFS
Frontend	Leaflet 1.9.4, HTML, CSS, JavaScript	Collects input and displays results

When the user presses Find services, the page sends a WFS GetFeature request to GeoServer with the search values in the viewparams parameter. GeoServer inserts these values into a stored SQL view, PostGIS runs the query, and GeoServer returns the answer as GeoJSON. Leaflet then draws the markers and fills the result list.

3. Data

The data was downloaded from the Overpass API with a Python script (load_pois.py) and inserted into PostGIS. The study area is central Pune, defined by the bounding box 18.40, 73.72, 18.67, 74.00 (south, west, north, east).

Eight categories are included: hospital, clinic, pharmacy, police, ATM, school and fuel (from the OpenStreetMap amenity tag) and park (from the leisure tag). In total 2,955 places were loaded. Features that are polygons in OpenStreetMap, such as large hospitals and parks, are reduced to a single centre point so that every record is a point.

The subcategory is taken from the healthcare speciality, operator type or brand tag when one exists, and is set to general otherwise. OpenStreetMap does not tag these fields consistently, so many records have the value general.

4. PostGIS Database

All places are stored in one table named poi.

Column	Type	Meaning
id	serial, primary key	Internal identifier
osm_id	text, unique	OpenStreetMap type and id, used to avoid duplicates
name	text	Place name
category	text	Service type
subcategory	text	Optional finer type
address, phone, website, opening_hours	text	Detail fields shown in the panel
geom	geometry(Point, 4326)	Location

Coordinate reference system. Locations are stored in EPSG:4326 (WGS 84 longitude and latitude), the same system used by GPS and web maps. Degrees are not a unit of distance, so the queries cast the geometry to the geography type. PostGIS then measures on the Earth spheroid and returns distances in metres. An alternative would be to project the data to EPSG:32643 (UTM zone 43N, which covers Pune) and measure in planar metres. The geography approach was chosen because it needs no reprojection and stays accurate for any location.

Indexes. A GiST index on geom, a GiST index on the geography expression, and a B-tree index on category and subcategory keep the queries fast.

5. Spatial Query

The query used by the application is:

```
SELECT id, name, category, subcategory, address, phone, website, opening_hours,
  ST_Y(geom) AS y_lat, ST_X(geom) AS x_lng,
  ST_Distance(geom::geography,
    ST_SetSRID(ST_MakePoint(%lng%,%lat%),4326)::geography) AS dist_m,
  geom
FROM poi
WHERE category = ANY(string_to_array('%cats%','|'))
  AND ('%sub%' = 'all' OR subcategory = '%sub%')
  AND ST_DWithin(geom::geography,
    ST_SetSRID(ST_MakePoint(%lng%,%lat%),4326)::geography, %radius%)
ORDER BY dist_m
LIMIT %n%
```

ST_DWithin keeps only places inside the search radius and can use the spatial index. ST_Distance gives the exact distance in metres, and ORDER BY dist_m sorts nearest first. LIMIT keeps only the nearest N results. The values written between percent signs are parameters filled in by GeoServer.

The three search modes use the same query:

- **Radius plus N:** the radius and the limit are both applied.
- **Radius only:** the limit is set to a large number (500) so every place in the radius is returned.
- **Nearest N only:** the radius is set to a very large value (100 km) so the limit decides the result.

6. GeoServer Layers

A workspace named services and a PostGIS data store named gis_db connect GeoServer to the database. Two layers are published.

1. **services:poi** is the full table, published as an ordinary layer. It is served as a WMS layer and can be switched on from the layer control on the map to show every service point.
2. **services:nearby_pois** is a SQL view layer that holds the spatial query above. The frontend requests it through WFS and passes the viewparams values lat, lng, radius, n, cats and sub.

Each SQL view parameter has a default value and a validation regular expression. Coordinates and numbers must match a numeric pattern, and the category and subcategory values may only contain letters, digits, underscores and spaces. Any other input is rejected by GeoServer, which protects the database from SQL injection.

In the GeoJSON output of the SQL view, GeoServer reports the geometry as null. To work around this, the query also returns the coordinates as the ordinary attributes y_lat and x_lng, and the frontend uses those to place the markers.

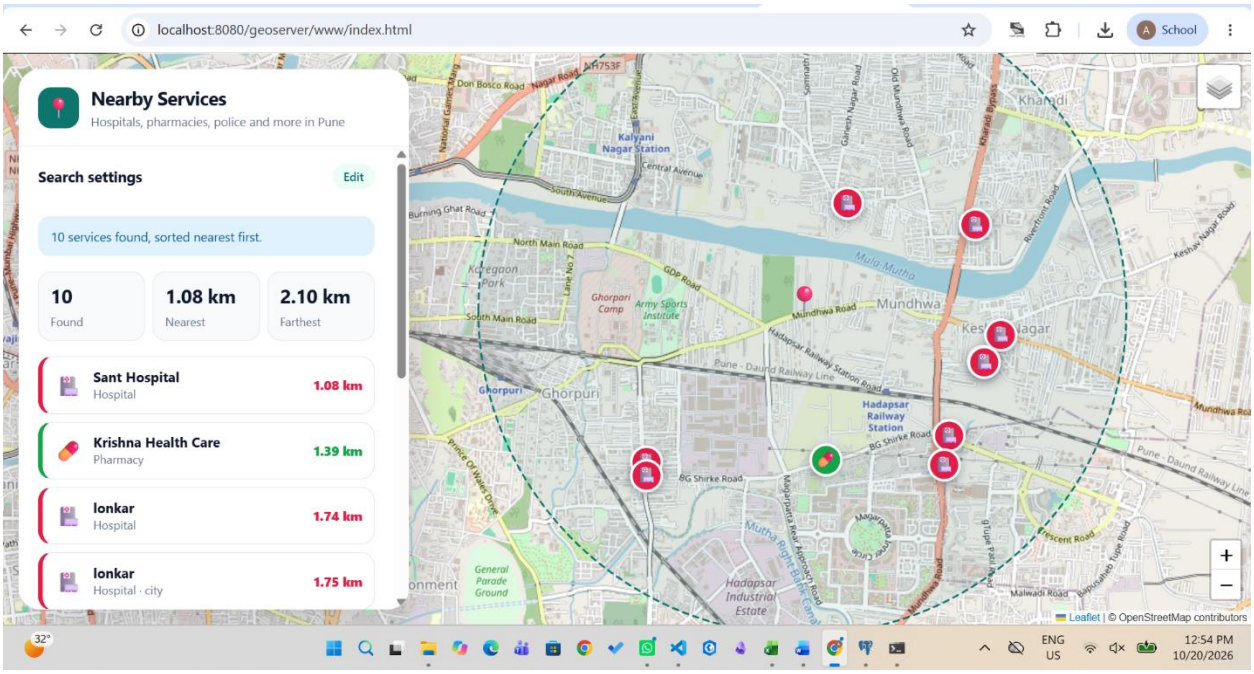
GeoServer applies a Content Security Policy to pages in its static folder. The policy was adjusted to allow map tiles from tile.openstreetmap.org, and the web page is served from the GeoServer data directory so that all requests share the same origin.

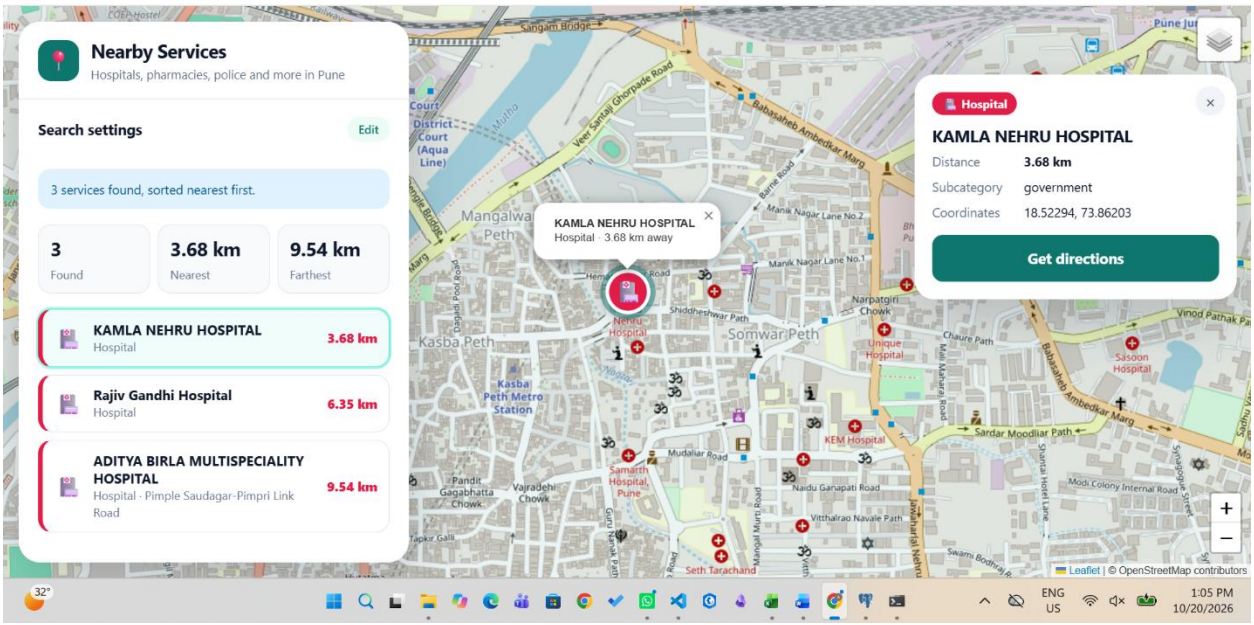
7. Frontend

The frontend is a single page built with Leaflet. The map fills the screen, and a floating panel holds the search controls and the results.

- **Location input:** click on the map, type latitude and longitude, or use the browser geolocation button.
- **Categories:** eight colour-coded category buttons, plus a subcategory list filled from the database.
- **Search mode:** radius plus N, radius only, or nearest N only, with a radius slider and a number-of-results field.
- **Results:** a list sorted by distance, with a summary of how many places were found and the nearest and farthest distances.
- **Map synchronisation:** clicking a card highlights its marker and opens its popup. Clicking a marker highlights and scrolls to its card.

- **Details panel:** name, category, subcategory, distance, address, phone, website, opening hours, coordinates and a link for directions.
- **Published layer:** the GeoServer WMS layer can be toggled in the layer control.





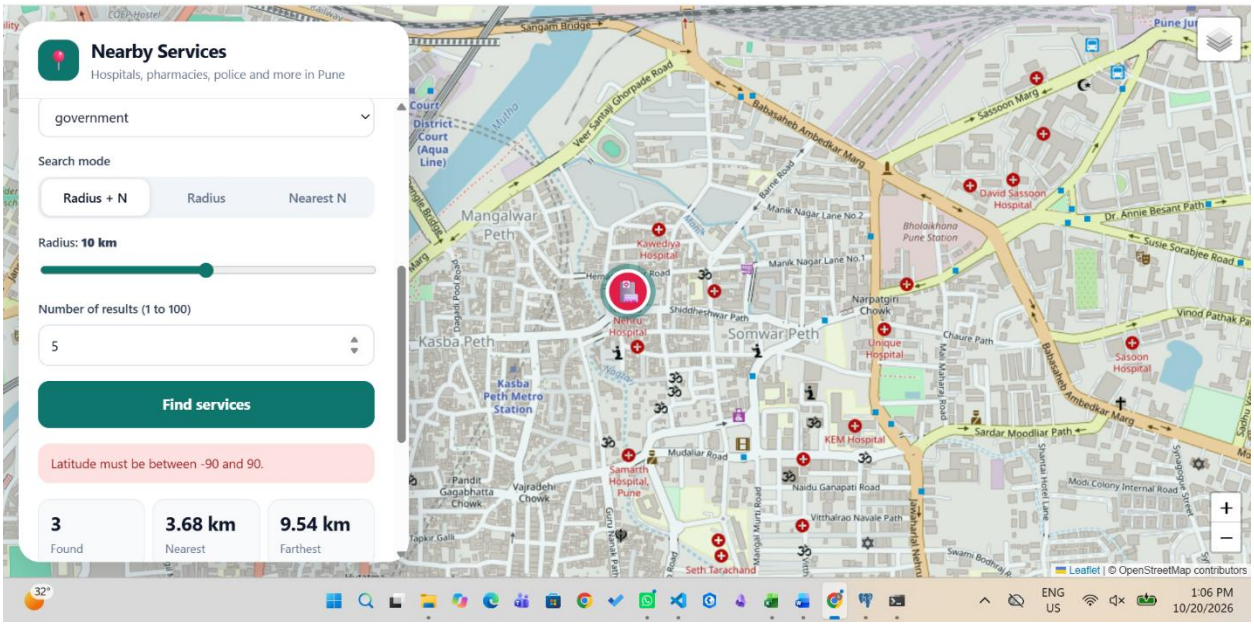
8. Requirements Checklist

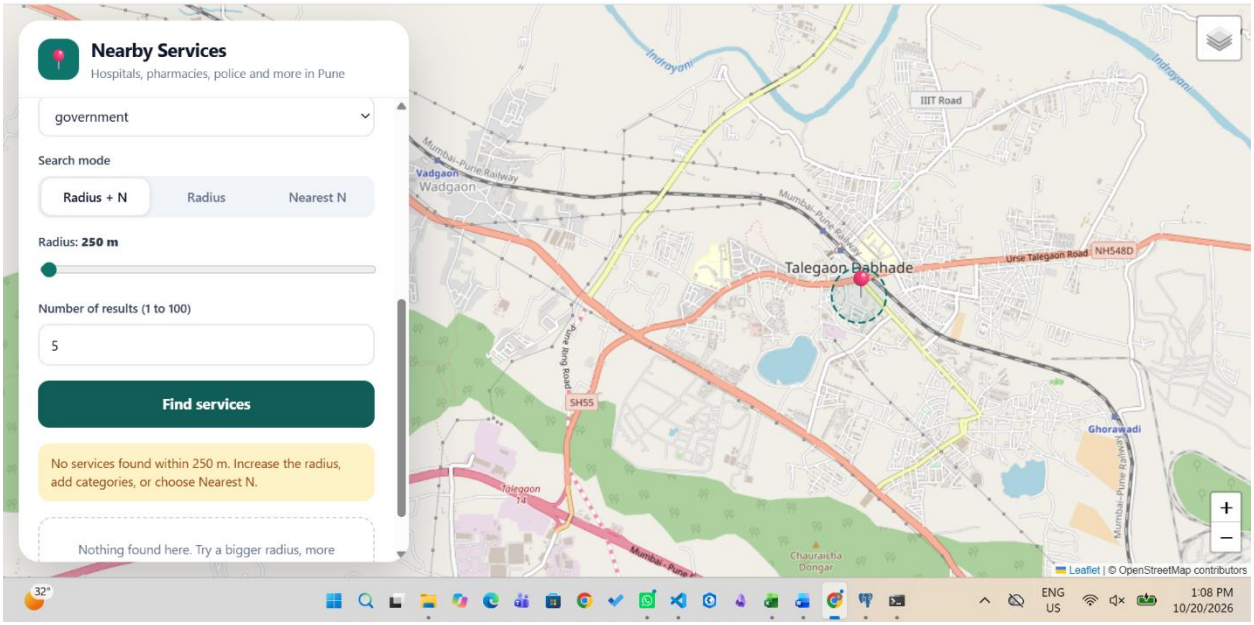
Requirement	How it is met
Location by map click, latitude and longitude, geolocation	All three inputs are available

Requirement	How it is met
Category and optional subcategory filter	Category buttons and a subcategory list
Radius and nearest-N search	Three search modes
Distance-sorted list synchronised with markers	Sorted by dist_m, linked in both directions
Popup and side-panel information	Popup on each marker and a details panel
PostGIS query with CRS and distance handling	EPSG:4326, geography cast, ST_DWithin and ST_Distance in metres
Layers published through GeoServer and a Leaflet frontend	WMS layer poi, WFS SQL view nearby_pois, Leaflet page
Error handling	Described in the next section

9. Error Handling

- **Denied or failed geolocation:** separate messages for permission denied, position unavailable and timeout. The message tells the user to click the map or type coordinates instead.
- **Invalid input:** the page checks that latitude and longitude are numbers within range (latitude from -90 to 90, longitude from -180 to 180), that at least one category is selected, and that the number of results is a whole number from 1 to 100.
- **No results:** a warning explains that nothing was found and suggests a larger radius, more categories or the nearest-N mode.
- **Server problems:** if GeoServer cannot be reached or returns an error, the page shows a clear message. A search that takes longer than 20 seconds is stopped with a timeout message.





10. Testing

Test	Expected result
Click the map and search for hospitals and pharmacies	Markers and a distance-sorted list appear
Type coordinates and search	The marker moves and results update
Deny the geolocation permission	A permission-denied message appears
Enter latitude 95	An out-of-range error appears
Search with Nearest N only from a distant point	The N nearest places are returned
Choose a subcategory	Only matching places are returned
Search with a very small radius in an empty area	A no-results warning appears
Stop GeoServer and search	A cannot-reach-GeoServer message appears

The database queries were also run directly in pgAdmin to confirm that the distances and ordering match what the web page shows.

11. Limitations and Future Work

- The data covers central Pune only. Other cities can be added by changing the bounding box in the loading script and running it again.
- Subcategories depend on OpenStreetMap tags, and many places only have the value general.
- Opening hours come from OpenStreetMap and are missing for many places. The distance is measured in a straight line, not along roads.
- The system runs on a local computer. Public use would need a server and secure web hosting.

- Possible improvements include routing along roads with a tool such as pgRouting, live open-now status, and marker clustering for large result sets.

12. Conclusion

The project delivers a working WebGIS that meets the stated requirements. PostGIS performs the spatial search with correct metre-based distances, GeoServer publishes the data through standard OGC services, and a Leaflet interface lets users choose a location, filter by category, search by radius or nearest N, and inspect each service on the map.

Appendix: How to Run

1. Install PostgreSQL with PostGIS, and GeoServer.
2. Create a database named gis and run CREATE EXTENSION postgis.
3. Restore the data from database/poi_dump.sql.
4. In GeoServer create the workspace services and the store gis_db, publish the poi table, and create the SQL view nearby_pois.
5. Copy the files in the frontend folder to the GeoServer www folder.
6. Open http://localhost:8080/geoserver/www/index.html in a browser.